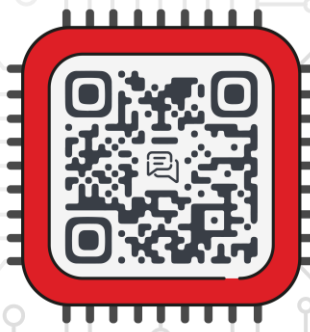




7

26

3120 sp

نظري

كلية الهندسة المعلوماتية

السنة الثانية

Swing

د. محمد الأحمد



RB Informatics; 8/1/2025

برمجة ٣

أهلاً بكم أعزائنا الطلاب في المحاضرة الأخيرة من مقر البرمجة 3

AWT and Swing:

Java provides two sets of components for GUI programming:

- AWT: classes in the java.awt package
- Swing: classes in the javax.swing package

Abstract Window Toolkit (AWT):

دعونا نتذكر سوياً أهم النقاط في AWT

The Abstract Window Toolkit is a portable GUI library.

تعتبر AWT أول مكتبة تعاملت معها برامج الجافا بالGUI، وهي تعتمد على المكونات الأولى لنظام التشغيل، ويمكن استخدامها عبر جميع أنظمة التشغيل المختلفة التي تدعم جافا .

AWT provides the connection between your application and the native GUI.

تعمل AWT كحلقة وصل تربط بين تطبيقك وواجهة المستخدم الأصلية للنظام الذي يعمل عليه التطبيق، أي تتفاعل مع مكونات النظام المحلي لتوفير واجهة مستخدم تتماشى مع مظهر النظام.

AWT provides a high-level abstraction since it hides you from the underlying details of the GUI your program will be running on.

AWT توفر تجزيراً عالياً، أي أن التفاصيل الدقيقة المتعلقة بكيفية عمل الواجهة ليس من الضروري أن يعلمها المبرمج فتقوم AWT بإخفائها عنه.

AWT components depend on native code counterparts (called peers) to handle their functionality. Thus, these components are often called heavyweight components

بما أن AWT تعتمد على الكود الأصلي (الخاص بنظام التشغيل)، فإن المكونات التي يتم إنشاؤها باستخدامه تعتبر مكونات ثقيلة، وهذا يعني أنها تتطلب المزيد من الموارد وبالتالي فإن سرعة الأداء أقل مقارنة ب swing

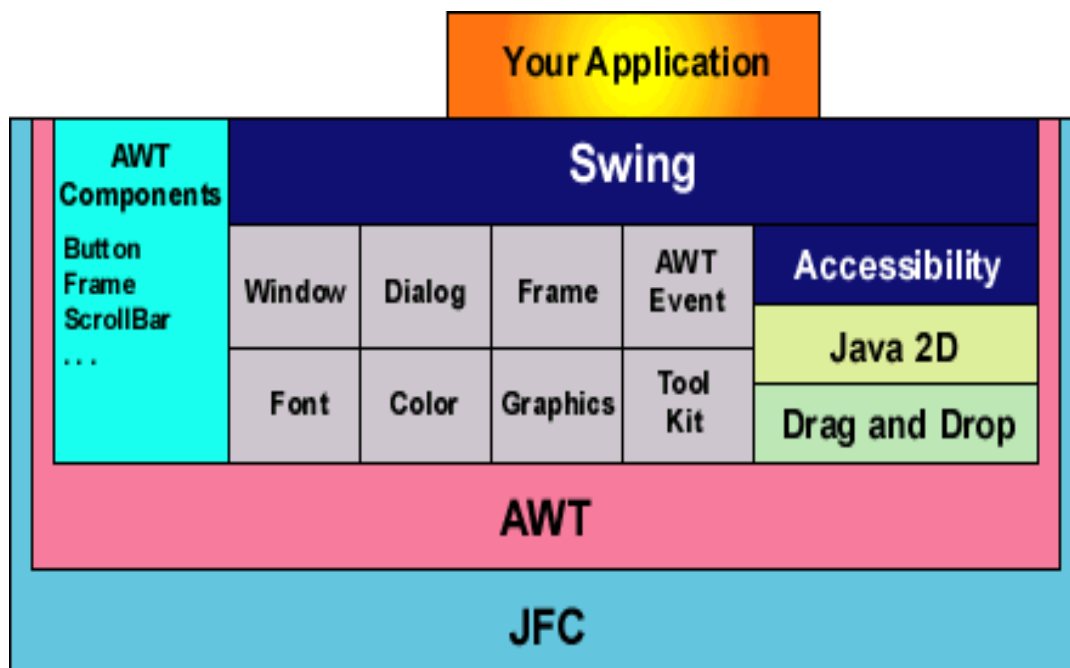
ملاحظة : peers هي مكونات تمثل الكود الأصلي لنظام التشغيل، ومكونات AWT تعتمد عليها

Swing:

Swing implements GUI components that build on AWT technology. ■
هي إطار عمل لتطوير واجهات المستخدم الرسومية، تم تطويرها كجزء من AWT (بناءً عليها) وتتميز بالعديد من الخصائص.

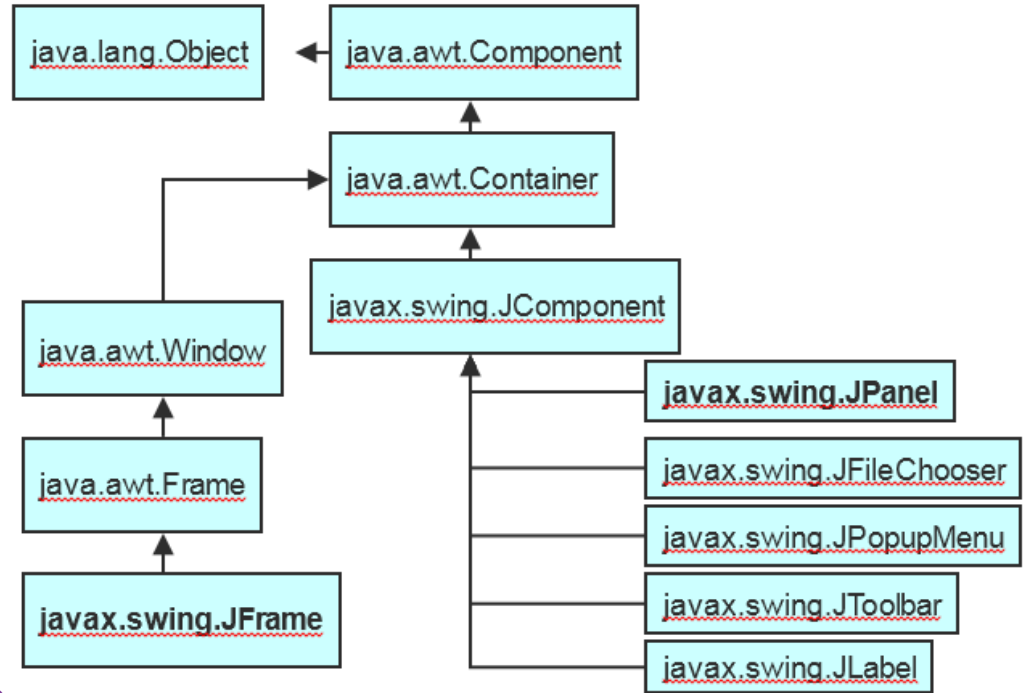
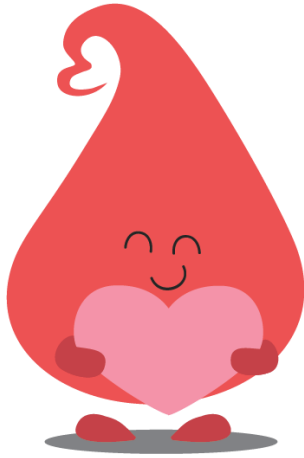
Swing is implemented entirely in Java. ■
أي أن جميع المكونات والوظائف الموجودة في Swing هي مكتوبة بالكامل في جافا، مما يجعلها مستقلة بحد ذاتها ومرنة أكثر.

Swing components do not depend on peers to handle their functionality. Thus, these components are often called lightweight components ■
مكونات الـ Swing تعتبر خفيفة الوزن (lightweight) لأنها لا تعتمد على المكونات الأصلية للنظام (peers) ويتم رسم المكونات وإدارتها بشكل كامل بواسطة جافا، مما يوفر مظهراً متسقاً عبر جميع أنظمة التشغيل.

Swing Stack:

Some AWT and Swing Classes :

يوضح المخطط التالي بعض الكلاسات المستخدمة ضمن AWT وSwing وعلاقاتها:

AWT & Swing: Pros and Cons:

Swing	AWT	
Portability: Pure Java implementation. بما أن Swing هو إطار عمل مبني بلغة جافا فهذا يعني أنه يمكن تشغيله على أي نظام تشغيل يدعم جافا، وهذا ما يجعل التطبيقات قابلة للنقل بسهولة بين أنظمة التشغيل المختلفة دون الحاجة إلى تعديل الأكواد، مما يوفر الوقت والجهد في تطوير التطبيقات. Features: Not limited by native components. توفر مجموعة واسعة من المكونات التي لا تعتمد على مكونات النظام الأصلية، أي يمكن إنشاء واجهات مستخدم غنية بالمكونات المتنوعة بدون قيود المكونات الأصلية .	Speed: native components speed performance. وذلك لأن مكونات الـ AWT تعتمد على مكونات النظام المحلي، أي أنها تستخدم موارد متاحة في نظام التشغيل مباشرة، فهي تستفيد من تحسينات الأداء الموجودة في نظام التشغيل وهذا يؤدي إلى أداء أسرع مقارنة بالمكونات التي تبني بالكامل في جافا مثل مكونات الـ Swing .	Pros إيجابيات
Look and Feel: Pluggable look and feel. Components automatically have the look and feel of the OS their running on. تدعم Swing مفهوم Pluggable look and feel ، مما يعني أن تغيير مظهر التطبيق يمكن بسهولة حيث تكون المكونات فيه تتكيف تلقائياً مع مظهر النظام الذي تعمل عليه.	Look and feel: AWT components more closely reflect the look and feel of the OS they run on. أي أن مكونات الـ AWT تعكس المظهر لنظام التشغيل الذي تعمل عليه، أي في حال استخدام الـ AWT على نظام ويندوز فستبدو المكونات مثل الأزرار وغيرها مشابهة لتلك الموجودة في تطبيقات الـ ويندوز الأصلية.	

Swing	AWT	
Performance: Swing components handle their own painting (instead of using APIs like DirectX on Windows). أي أنه في Swing كل مكون مثل الأزرار وغيرها يعتمد على الرسم الذاتي، ويتم ذلك من خلال أسلوب painting والذي يعتمد على رسم المكونات باستخدام كائنات graphics وهذا يعني أن لا يعتمد على برمجة التطبيقات الأصلية مثل DirectX على Windows وQuartzs على macOS	Portability: use of native peers creates platform specific limitations. أي أن استخدام المكونات الأصلية في النظام قد يتعرض لبعض القيود، فمن الممكن أن لا تكون متاحة على نظام تشغيل معين مما يجعل من الصعب على المطورين إنشاء تطبيقات متسقة عبر المنصات (صعوبة في نقل التطبيقات بين أنظمة التشغيل المختلفة)	Cons سلبيات
Look and Feel: May look slightly different than native components أي على الرغم من أن Swing تدعم ميزة look and feel واحد، إلا أن المكونات قد لا تبدو تماماً مثل المكونات الأصلية في بعض الأحيان، بحيث قد يشعر بعض المستخدمين بأن التطبيق ليس جزءاً من نظام التشغيل الخاص بهم.	Features: AWT supports only the lowest common denominator—e.g. no tool tips or icons. أي أن AWT تقدم مجموعة محددة من الوظائف مقارنة بـSwing، فعلى سبيل المثال AWT لا تدعم ميزات مثل تلميحات الأدوات (tool tips) أو الرموز (icons) بشكل افتراضي، وهذا ما يجعل التصميم والتفاعل في التطبيقات محدوداً وأقل جاذبية من غيرها.	

Main Steps in GUI Programming

- To make any graphic program work we must be able to create windows and add content to them, to make this happen we must:

لإنشاء نوافذ وإضافة محتوى عليها في أي تطبيق يجب اتباع الخطوات التالية:

1. Import the AWT or Swing packages.

استيراد مكتبات الـ swing أو الـ AWT ، ويُفضل استيرادها معاً، لأنه من الممكن استخدام بعض المكونات من swing واستخدام الـ graphics , colors, fonts من الـ AWT .

2. Set up a top-level container.

إنشاء نافذة أو إطار رئيسي لعرض المكونات.

3. Fill the container with GUI components.

بعد إنشاء container ، يمكننا إضافة مكونات واجهة المستخدم مثل الأزرار والنصوص.

4. Install listeners for GUI Components.

أي يجب تثبيت مستمعي الأحداث لمعالجة التفاعلات مع المستخدم.

5. Display the container.

أي يجب جعل النافذة أو الإطار مرئياً باستخدام الدالة (Boolean) `setVisible`، حيث تكون القيمة الافتراضية لها `false`، وهي دالة مهمة جداً من دونها لا يظهر أي مكون على شاشة المستخدم.

■ ملاحظة :

هناك دالة أيضاً مسؤولة عن إظهار المكونات وتعمل نفس عمل `setVisible` وتدعى `show`، وتستخدم مع `JFrame`، `JDialog`، ولكنها قديمة جداً وتم استخدام `setVisible` بدلاً منها، ولا يمكن استخدام الدالتين `show` و `setVisible` معاً.

Hello World Example:

```
1 import javax.swing.*;
2 public class main {
3     public static void main(String[] args) {
4         // إنشاء كائن frame وهو يمثل نافذة التطبيق، وتعيين العنوان
        HelloWorldSwing
5         JFrame frame = new JFrame("HelloWorldSwing");
6         // إنشاء كائن من JLabel، وهو عنصر واجهة مستخدم يُستخدم لعرض
        نص لا يمكن التعديل عليه (تم تعريفه كـ final ولكن هو تلقائياً يعرض نصاً
        ثابتاً، وفي مثالنا (Hello World))
7         final JLabel label = new JLabel("Hello World");
8         // إضافة العنصر label إلى محتوى النافذة (frame).
        getContentPane() يُستخدم للحصول على اللوحة الأساسية التي تحتوي
        على جميع العناصر المضافة إلى النافذة.
9         frame.getContentPane().add(label);
10        // إنهاء البرنامج بالكامل (الخروج من التطبيق) عند إغلاق النافذة
11        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
12        // ضبط حجم النافذة تلقائياً بناءً على حجم العناصر، وهنا تأخذ الحجم
        المناسب لعرض label
13        frame.pack();
14        // جعل النافذة مرئية للمستخدم، إذا لم يتم استدعاء هذه الميثود فلن
        تظهر النافذة حتى لو تم إنشاؤها.
15        frame.setVisible(true);
16    }
17 }
```

Output:



Top-level Containers

هي نوع من الحاويات (containers) التي تمثل المستوى الأعلى في هيكل واجهة المستخدم، وهذه الحاويات تكون مستقلة، ويمكن أن تحتوي على مكونات أخرى مثل الأزرار والحقول النصية وغيرها، وكل حاوية من الممكن أن تحتوي على عنصر واحد من نوع الـ content pane (panel) والذي يستخدم لترتيب وتنظيم المكونات داخل الحاوية، وهناك ثلاثة أنواع رئيسية من Top-level Containers سنتعرف عليها الآن :

There are three top-level Swing containers:

1. JFrame:

window that has decorations, such as a border, a title, and buttons for iconifying and closing the window

هو الإطار الرئيسي في واجهة المستخدم، وهي مستقلة بحد ذاتها للتفاعل مع المستخدم، وتحتوي على decorations مثل الحدود والعنوان وأزرار التحكم (إغلاق - تكبير وتصغير - استعادة). ويمكن تعيين عنوان للنافذة عن طريق تمرير العنوان كباراميتير للـ constructor، وفي حال تركه فارغاً فإن الـ JFrame ستظهر من دون عنوان، ويمكن أيضاً تحديد سلوك الإطار عند الإغلاق.

إنشاء نافذة من JFrame بعنوان swing → `JFrame frame=new JFrame("swing");`

وكما يمكن إنشاء Frame خاص بنا عن طريق إنشاء كلاس وجعله يرث من الكلاس JFrame، وذلك يساعد في تنظيم الكود وسهولة الوصول إلى عناصر الواجهة.

2. JDialog:

هي نافذة ثانوية تعتمد على نافذة أخرى مستقلة (JFrame)، وتستخدم لعرض حوارات أو نوافذ منبثقة مثل رسائل التأكيد أو إدخال البيانات من المستخدم وهي طريقة غير فعالة نوعاً ما.

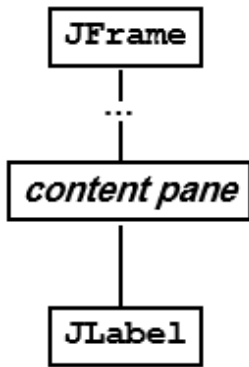
3. JApplet:

applet's display area within a browser window

هي حاوية تستخدم لتشغيل تطبيقات صغيرة (applet's) داخل متصفح الويب.

Containment Hierarchy:

نلاحظ أنه في مثال Hello World السابق تم استخدام content pane وسنلاحظ أنه في أغلب الأمثلة سيتم استخدامه أيضاً، لكن ما الفائدة منه؟



content pane: هي حاوية وسيطة (an intermediate container) تستخدم لتجميع جميع المكونات المرئية في نافذة التطبيق، وكل حاوية رئيسية (top-level container) مثل JFrame تحتوي على هذه الـ panel الافتراضية (في حالة أن المبرمج لم يقم بإنشاء panel في الإطار فهي موجودة تلقائياً). تساعد هذه الـ panel بشكل مباشر أو غير مباشر (directly or indirectly) في تنظيم المكونات المرئية (visible) داخل النافذة، مما يسهل إدارتها وإمكانية التعديل عليها.

ولا بد من إضافة المكونات إلى حاويات ليتم إظهارها وللإضافة يتم استخدام الميثود add بأشكالها المختلفة، بما في ذلك إضافة الـ panel إلى النافذة أيضاً.

ملاحظة :

- من الممكن أن تحتوي panel (وهي تعتبر sub container) على panel أخرى بداخلها ولكن لا يمكن أن يكون هناك إطار JFrame داخل إطار آخر.
- ال panel لا تحتوي على أي عنوان ولا أي من أزرار التحكم.
- لتعريف panel في Swing : `JPanel panel = new JPanel ();`

تساؤل ! :

ما الهدف من المكونات الموجودة في swing & awt ؟

- JButton: الهدف منها هو إحداث action واحد عند النقر عليه، ويكون enabled or disabled. لإنشائه: `JButton b = new JButton (" ");` النص الذي سيظهر للمستخدم فوق الزر
 reference للتعامل مع الزر كobject
- JTextField: إدخال نص حصراً، وهو عبارة عن سطر واحد فقط.
- JTextArea: إدخال نص ولكن يتكون من عدة سطور.
- JCheckBox: يتيح للمستخدم اختيار (أو إلغاء اختيار) خيار معين، ويمكن للمستخدم تحديد خيارات متعددة في نفس الوقت.
- JRadioButton: يسمح للمستخدم باختيار خيار واحد من مجموعة من الخيارات.
- JComboBox: يتيح للمستخدم اختيار عنصر واحد فقط من القائمة المنسدلة.
- JPasswordField: إدخال نص ولكنه مخفي (كلمة مرور)، مما يمنع أي شخص من رؤية كلمة المرور أثناء الكتابة، ويتم عرض رموز بدلاً من النص الفعلي (نجوم أو نقاط).
- JLabel: عرض نص ثابت وغير قابل للتعديل (للإظهار مثل أدخل البريد الإلكتروني ..) ولا يوجد Event خاصة به.

Handling Events:

- Every time the user types a character or pushes a mouse button, an event occurs.

في Java ، كلما قام المستخدم بكتابة حرف أو ضغط على زر الماوس يتم إطلاق حدث (Event) ، هذا يسمح للتطبيقات بالتفاعل مع المستخدم بطريقة ديناميكية.

- Any object can be notified of the event.

يتم التعامل مع الأحداث باستخدام مفهوم المستمعين Listeners وهم أشبه بمراقبين ينتظرون حدوث الحدث ويتصرفون استناداً على ما تم رصده، حيث يمكن لأي كائن Listener تلقي إشعار عند حدوث event معين.

- All the objects has to do is implement the appropriate interface and be registered as an event listener on the appropriate event source.

ويتم تنفيذ هذا المفهوم من خلال استخدام الانترفيس الموجود في جافا والمناسب للحدث، والذي يحوي على توابع يجب القيام بـ override لها عند الحاجة ، وربطه مع مصدر الحدث (button, text field...).

How to Implement an Event Handler:

Every event handler requires three pieces of code:

1. Declaration of the event handler class that implements a listener interface or extends a class that implements a listener interface.

إنشاء كلاس لمعالجة الحدث يقوم بتنفيذ انترفيس المستمع أو يرث من كلاس يقوم بذلك، مثلاً:

```
public class MyClass implements ActionListener{
```

2. Registration of an instance of the event handler class as a listener.

إنشاء كائن من الكلاس المعالج للحدث (في مثالنا MyClass) وإضافته كمستمع للحدث الذي من الممكن أن يحدث على أحد المكونات (سندعوه في مثالنا someComponent)، ويمكن أن نضع "this keyword" بدلاً عن الكائن كـ argument للدالة.

```
MyClass instanceOfMyClass = new MyClass();
```

```
someComponent.addActionListener(instanceOfMyClass);
```

3. providing code that implements the methods in the listener interface in the event handler class

وأخيراً نقوم بإعادة تعريف (override) للدالة المناسبة، وفي مثالنا هي الدالة actionPerformed الرئيسية في الانترفيس ActionListener.

```
public void actionPerformed(ActionEvent e) { ...//code that reacts to the action... }
```

```

1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.ActionEvent;
4 import java.awt.event.ActionListener;
5 // وراثته الكلاس JFrame وتنفيذ الانتريفييس
ActionListener
6 //declares a class that implements a
listener interface (i.e. ActionListener)
7 public class main extends JFrame
implements ActionListener {
8
9 JButton b = new JButton("Click me!");
10 public main() {
11 // ربط المستمع مع المكون
12 // registers an instance of this class
with the event source
13 b.addActionListener(this);
14 // هذه الدالة تستخدم للوصول الى المنطقة التي
يمكننا فيها اضافة مكونات ال panel باستخدام الدالة
add
15 getContentPane().add(b);

```

```

16 pack();
17 setVisible(true);
18 }
19 تحديد الإجراء الذي يحدث عند الضغط على الزر
20 // defines the action to take when the
event occurs
21 public void actionPerformed
(ActionEvent e) {
22 عند النقر على الزر، يتم تغيير لون خلفية
الزر إلى اللون CYAN
23 b.setBackground(Color.CYAN);
24 }
25 public static void main(String[] args) {
26 عند تشغيل البرنامج يتم إنشاء كائن جديد
من main
27 new main();
28 }}

```

Output:



■ الخلاصة : لكي يتفاعل كائن ما مع الأحداث يجب عليه :

- 1- تنفيذ الـ interface المعالج للحدث.
- 2- التسجيل كمستمع على مصدر الأحداث.
- 3- استدعاء الدالة المناسبة لمعالجة الحدث.

Action	Event
هو تغيير بـ state، ويشير إلى تفاعلات المستخدم، مثل النقر على الأزرار، تحريك الفأرة، تصغير وتكبير، اختيار خيار من check box أو radio button، أو إدخال بيانات في حقول النص، وغيرها الكثير...	يشير إلى ما يحدث عند الاستجابة لـ action معين (نتيجة له)، مثل تنفيذ الكود في ميثود actionPerformed عند حدوث ActionEvent (ضغط زر)، وهذا ما يدعى بالـ event handling. أنواع الأحداث: - ActionEvent: يحدث عند تنفيذ إجراء، مثل الضغط على زر. - MouseEvent: يحدث عند تفاعل المستخدم مع الفأرة، مثل النقر أو الحركة. - KeyEvent: يحدث عند الضغط على مفتاح من لوحة المفاتيح.

الدالة actionPerformed هي جزء من الـ ActionListener ، وهي abstract لذا لا بد من القيام بـ override لها، وتستخدم للتعامل مع الأحداث حيث تستقبل كائناً من نوع(ActionEvent) يحتوي على معلومات حول الحدث الذي وقع، ويتم تنفيذ التعليمات التي بداخلها عندما يحدث الحدث مثل الضغط على زر أو ...، وهي أهم ميثود في الـ ActionListener.

ملاحظة أريسية: يمكن استخدام مفهوم lambda expression في هذا الموضع (مع الـ ActionListener) وهي ميزة في الإصدار 8 من جافا، وتسمح بكتابة تعبيرات مختصرة تعبر عن وظائف دون الحاجة إلى إنشاء كائن جديد مرة أخرى، وهذا ما يجعل الكود أبسط ويقلل عدد الأسطر اللازمة لكتابة الكود.

Button.addActionListener(e -> System.out.println("Button clicked!"));

وهذا بدلاً من كتابة كل هذه الأسطر:

```
Button.addActionListener (new ActionListener() {
@Override
public void actionPerformed(ActionEvent e){
System.out.println("Button clicked!");}});
```

■ مثال :

```
1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.ActionEvent;
4 import java.awt.event.ActionListener;
5
6 //JFrame من ميراث main جعل الكلاس
7 public class main extends JFrame {
8     //هذا متغير ثابت يستخدم للنص المعروض
9     //في JLabel
10    private static String labelPrefix =
11    "Number of button clicks: ";
12    //تعريف متغير يحتفظ بعدد النقرات على الزر. يبدأ
13    //من 0
14    private int numClicks = 0;
15    //إنشاء كائن JLabel يسمى label ، والذي
16    //سيعرض عدد النقرات. يبدأ النص بـ "Number of
17    //button clicks: 0".
18    JLabel label = new JLabel(labelPrefix
19    + "0 ");
```

```
14 //أخذ العنوان الجديد وتحويله الى كونسرتكتور الاب
15 public main(String title) {
16     super(title);
17     //إنشاء زر جديد بعنوان "I'm a Swing
18     //button!";
19     //إضافة حدث بحيث عند الضغط على الزر،
20     //يتم استدعاء طريقة actionPerformed
21     button.addActionListener(new
22     ActionListener() {
23         //يتم زيادة قيمة numClicks بمقدار 1
24         //يتم تحديث نص label ليعكس العدد الجديد
25         //من النقرات
26         public void actionPerformed
27         (ActionEvent ae) {
28             label.setText(labelPrefix +
29             ++numClicks);
30         }
31     });
```

```

28      // انشاء panel لتجميع العناصر
29      JPanel panel = new JPanel();
30      // اضافة الزر وال label
31      panel.add(button);
32      panel.add(label);
33
34      getContentPane().add(panel);
35      // ضبط حجم النافذة تلقائياً بناءً على حجم
    العناصر المضافة وجعل النافذة مرئية للمستخدم
36      pack();
37      setVisible(true);
38  }
39  public static void main(String[] args){
40      new main("SwingApplication");
41  }
42  }

```

Output:



ImageIcon:

- Some Swing components can be decorated with an icon—a fixed-size image.

بعض مكونات Swing مثل (الأزرار، والخلايا في الجداول، وغيرها) تتيح استخدام الايقونات مثل ImageIcon، لتزيين وتجميل الواجهة، وهي صورة ذات حجم ثابت.

- A Swing icon is an object that adheres to the **Icon** interface.

إضافة أريسيزية: يحوي Icon interface: الدوال:

- getWidth(): تُرجع عرض الرمز (الأيقونة).

- getHeight(): تُرجع ارتفاع الرمز.

- paintIcon(Component c, Graphics g, int x, int y): ترسم الرمز في نقطة معينة على الشاشة.

- Swing provides a particularly useful implementation of the Icon interface: ImageIcon.
- ImageIcon paints an icon from a GIF or a JPEG image

يمكننا استخدامه لتحميل وعرض الصور من ملفات صورة بامتدادات مثل JPEG أو GIF أو jpg

يمكننا استخدام ImageIcon مع عدة مكونات مثل JButton و JLabel

```

1 import javax.swing.*;
2 public class ImageIconExample extends JFrame {
3     public static void main(String[] args) {
4         JFrame frame = new JFrame("ImageIcon Example");
5         // إنشاء صورة باستخدام ImageIcon ويجب التأكد من أن
6         // الصورة موجودة في مكان تشغيل البرنامج (اللاحقة . اسم الصورة)
7         // أو نقوم بوضع مسار التخزين للصورة مثل ("path/to/image.gif")
8         ImageIcon icon = new ImageIcon("smallfrog.jpg");
9         JPanel panel = new JPanel();
10        // جعل الزر يحمل الصورة التي تم تحميلها
11        JButton button = new JButton(icon);
12        panel.add(button);
13        frame.getContentPane().add(panel);
14        frame.pack();
15        frame.setVisible(true); } }

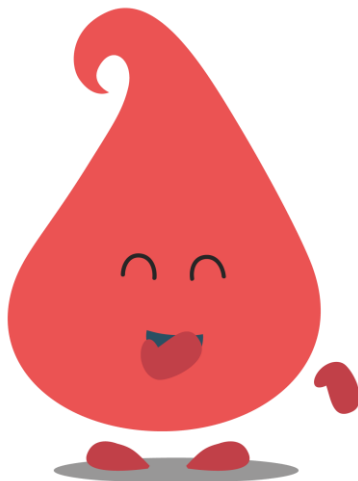
```

Output:



JTextField:

هو عنصر واجهة مستخدم يسمح للمستخدم بإدخال نص من سطر واحد، ويمكن استخدامه لأغراض متعددة مثل إدخال اسم أو عنوان أو أي معلومات نصية أخرى، يمكن تحديد عدد الأحرف التي يمكن إدخالها ويمكن استخدام DocumentListener للاستجابة لتغييرات النص في الحقل، ويمكن إعطاء الحقل نصاً افتراضياً.



Every day is another chance
to change your life.

ملاحظات :

(1) JTextField يتعامل مع النصوص فقط لذا يُمكن إدخال مثلاً 0 كنص ومن ثم نحتاج إلى تحويله إلى عدد، ويمكن أن يتم ذلك باستخدام طريقة parseInt() أو parseDouble()

(2) أي component يجب تحديد الـ size الخاص به بحيث يتناسب مع الـ panel الموجود بداخله .

(3) إن عدد الـ columns الافتراضي هو 0 ويمكن تغييره حسب حاجة المستخدم، وهو يعبر عن عدد الخانات التي يمكن فيها كتابة النص، مثلاً:

```
JTextField namefield = new JTextField(0);
```

هنا سيظهر بهذا الشكل:  ويمكن تغييرها مثلاً لتستوعب 10 خانات :

```
JTextField namefield = new JTextField(10);
```

(4) يحوي الكلاس على الميثود getText() التي ترجع النص المدخل في الحقل والميثود setText(String text) التي تعيّن فيه نصاً جديداً.

■ مثال:

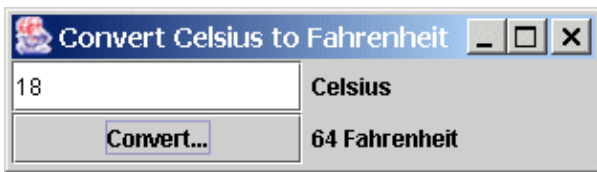
```
1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.ActionEvent;
4 import java.awt.event.ActionListener;
5 public class CelsiusConverter
6 implements ActionListener
7 {
8     JFrame converterFrame;
9     JPanel converterPanel;
10    // حقل نصي لإدخال درجة الحرارة المئوية
11    JTextField tempCelsius;
12    // تسميات لعرض النص
13    JLabel celsiusLabel, fahrenheitLabel;
14    // زر لتحويل درجة الحرارة
15    JButton convertTemp;
16    public CelsiusConverter() {
17        converterFrame = new JFrame
18        ("Convert Celsius to Fahrenheit");
19        converterPanel = new JPanel();
```

```
19    // إنشاء JPanel وجعلها بصفين
20    // وعمودين باستخدام GridLayout
21    converterPanel.setLayout(new
22    GridLayout (2, 2));
23    // استدعاء الدالة addWidgets() لإضافة المكونات إلى
24    // panel
25    addWidgets();
26    // إضافة الـ panel إلى الـ converterFrame
27    // وجعلها في وسط الإطار
28    converterFrame.getContentPane().add
29    (converterPanel, BorderLayout.CENTER);
30    converterFrame.setDefaultCloseOperation
31    (JFrame.EXIT_ON_CLOSE);
32    converterFrame.pack();
33    converterFrame.setVisible(true);
34    }
35    private void addWidgets() {
36        // إنشاء حقل نصي (JTextField) لإدخال درجة
37        // الحرارة، مع عرض 2 حرف
38        tempCelsius = new JTextField(2);
```

```

32 // إنشاء JLabel لعرض نص "Celsius" مع
    محاذة للييسار
33 celsiusLabel = new JLabel
    ("Celsius", SwingConstants.LEFT);
34 convertTemp = new JButton
    ("Convert...");
35 fahrenheitLabel = new JLabel
    ("Fahrenheit", SwingConstants.LEFT);
36 // إضافة مستمع حدث للزر بحيث عندما يتم
    الضغط عليه، سيتم استدعاء الطريقة
    actionPerformed
37 convertTemp.addActionListener(this);
38 converterPanel.add(tempCelsius);
39 converterPanel.add(celsiusLabel);
40 converterPanel.add(convertTemp);
41 converterPanel.add(fahrenheitLabel);

```



```

42 }public void actionPerformed
    (ActionEvent event) {
43 // الحصول على النص من tempCelsius
    وتحويله إلى Double
44 // استخدام المعادلة لتحويل درجة الحرارة من
    مقياس سيليزي إلى فهرنهايت
45 // تحويل النتيجة إلى عدد صحيح (int).
46 int tempFahr = (int)
    ((Double.parseDouble
    (tempCelsius.getText())) * 1.8 + 32);
47 // تحديث النص في fahrenheitLabel
    ليظهر درجة الحرارة المحولة
48 fahrenheitLabel.setText(tempFahr + "
    Fahrenheit");
49 }
50 public static void main(String[] args) {
51 try {
52 UIManager.setLookAndFeel (UIManager.
    getCrossPlatformLookAndFeelClassName());
53 } catch (Exception e) {}
54 CelsiusConverter converter = new
    CelsiusConverter(); } }

```

JCheckBox:

هو مكون في واجهة المستخدم الرسومية (GUI) يُستخدم لإنشاء مربع اختيار (Checkbox) يتيح للمستخدم اختيار (أو إلغاء اختيار) خيار معين، ويمكن للمستخدم تحديد خيارات متعددة في نفس الوقت، ويمكننا من إضافة نص مع مربع الاختيار مما يسمح للمستخدم بفهم الوظيفة أو الخيار الذي يمثلها، بالإضافة إلى أنه يمكن تحديد جميع الخيارات أو بعض منها افتراضياً.

ملاحظة أريسية: يمكن ربط JCheckBox بمعالج أحداث (Event Handler) للاستجابة لتغييرات الحالة ويمكن استخدام ItemListener لهذا الغرض.

```

1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.ActionEvent;
4  import java.awt.event.ActionListener;
5  public class CheckBoxDemo extends
JPanel implements
6      ActionListener {
7      // إنشاء مربعات خيارات
8          JCheckBox chinButton;
9          JCheckBox glassesButton;
10         JCheckBox hairButton;
11         JCheckBox teethButton;
12         JButton goButton = new
JButton("Go!");
13     public CheckBoxDemo() {
14         // إنشاء مربعات الاختيار (JCheckBox) لكل
من العناصر الأربعة (الذقن، النظارات، الشعر،
الأسنان) وتعيينها لتكون محددة بشكل افتراضي
بإستخدام.
15         chinButton = new JCheckBox("Chin");
16         chinButton.setSelected(true);
17         glassesButton = new JCheckBox
("Glasses");
18         glassesButton.setSelected(true);
19         hairButton = new JCheckBox("Hair");
20         hairButton.setSelected(true);
21         teethButton = new JCheckBox
("Teeth");
22         teethButton.setSelected(true);
23         // إضافة (ActionListener) للزر goButton
بحيث يتم استدعاء الدالة actionPerformed() عند
الضغط على الزر
24         goButton.addActionListener(this);
25         // تعيين تخطيط ال panel إلى GridLayout مع
عدد غير محدد من الصفوف وعمود واحد
26         setLayout(new GridLayout(0, 1));

```

```

27         // إضافة جميع مربعات الاختيار والزر إلى ال panel
28         add(chinButton);
29         add(glassesButton);
30         add(hairButton);
31         add(teethButton); add(goButton); }
32     public static void main(String s[]) {
33         // إنشاء نافذة جديدة (JFrame) بعنوان
"CheckBoxDemo"
34         JFrame frame = new JFrame
("CheckBoxDemo");
35         frame.setDefaultCloseOperation
(JFrame.EXIT_ON_CLOSE);
36         // إضافة كائن من CheckBoxDemo إلى frame
37         frame.getContentPane().add(new
CheckBoxDemo());
38         frame.pack();
39         frame.setVisible(true);
40     }
41     // استدعاء الدالة actionPerformed
(ActionEvent e) عندما يتم الضغط على الزر
42     public void actionPerformed(ActionEvent e) {
43         // إذا قام المستخدم بتحديد glassesButton
طباعة Glasses = true
44         if (glassesButton.isSelected()) {
45             System.out.println("Glasses = true");
46             // إذا لم يتم طباعة Glasses = false
} else {
47                 System.out.println("Glasses = false");
48             }
49         // إنهاء البرنامج
50         System.exit(0); }

```



You may not want to be alerted every time the user selects or deselects a checkbox.

A more common use is to check the state of the button when the user clicks a button signifying that he/she is done and ready to advance

عند استخدام checkbox يفضل السماح للمستخدم بتعديل خياراته دون اظهار تنبيه عند كل تعديل على الخيارات، ويمكن ذلك من خلال انشاء زر (مثل تم) لتأكيد خياراته دفعة واحدة، وهذا ما يجعل عملية الاختيار والتأكيد سلسلة وفعالة وتقلل من الأخطاء.

JRadioButton Example:

هو عنصر واجهة المستخدم يسمح للمستخدم باختيار خيار واحد من مجموعة من الخيارات

- **ButtonGroup** ensures that only one radio button in the group can be selected at a time.
- أي يمكن تجميع عدة أزرار اختيار في مجموعة (ButtonGroup) بحيث يُسمح للمستخدم باختيار زر واحد فقط في أي وقت.
- **setSelected** sets initial state. (Good for defaults).
- يمكن تعيين زر واحد ليكون مختاراً بشكل افتراضي باستخدام **setSelected**، وفي حال اختار المستخدم خياراً سيصبح الخيار المحدد افتراضياً **unselected**.
- **isSelected** checks the state of the button.

تستخدم للتحقق من الحالة الحالية لزر الاختيار، وترجع **true** إذا كان الزر محدداً و **false** إذا لم يكن كذلك.

ملاحظة أريسية: يستخدم عادة مع **ActionListener** للاستجابة لتغييرات الاختيار.

ملاحظة : يجب ان تكون الخيارات الموجودة **unique** (غير مكررة)

```

2 import javax.swing.*;
3 import java.awt.*;
4 import java.awt.event.ActionEvent;
5 import java.awt.event.ActionListener;
6 public class RadioButtonDemo
7 extends JPanel implements
8 ActionListener {
9     // تعريف String لكل حيوان، سيتم استخدامها //
10    كعناوين لأزرار الاختيار
11    String birdString = "Bird";
12    String catString = "Cat";
13    String dogString = "Dog";
14    String rabbitString = "Rabbit";
15    String pigString = "Pig";

```

```

14 // إنشاء خمسة أزرار اختيار، كل منها يمثل حيواناً مختلفاً
15 JRadioButton birdButton = new
16 JRadioButton(birdString);
17 JRadioButton catButton = new
18 JRadioButton(catString);
19 JRadioButton dogButton = new
20 JRadioButton(dogString);
21 JRadioButton rabbitButton = new
22 JRadioButton(rabbitString);
23 JRadioButton pigButton = new
24 JRadioButton(pigString);
25 // إنشاء زر باسم "Go" للإجراء الذي سيقوم به المستخدم //
26 بعد اختيار حيوان
27 JButton goButton = new JButton("Go!");

```

```

22     public RadioButtonDemo() {
23         // يتم تعيين birdButton كزر مختار بشكل افتراضي
24         birdButton.setSelected(true);
25         // إنشاء مجموعة من الأزرار (ButtonGroup) لضمان أنه يمكن اختيار زر واحد فقط في نفس الوقت
26         ButtonGroup group = new ButtonGroup();
27         group.add(birdButton);
28         group.add(catButton);
29         group.add(dogButton);
30         group.add(rabbitButton);
31         group.add(pigButton);
32
33         // إضافة مستمع الحدث (ActionListener) للزر goButton ، مما يعني أنه عندما ينقر المستخدم على الزر، سيتم استدعاء الدالة actionPerformed
34         goButton.addActionListener(this);
35         // تعيين تخطيط الشبكة (GridLayout) لعرض الأزرار عمودياً في عمود واحد
36         setLayout(new GridLayout(0, 1));
37         add(birdButton);
38         add(catButton);
39         add(dogButton);
40         add(rabbitButton);
41         add(pigButton);
42         add(goButton);
43     }
44     public static void main(String s[]) {
45         JFrame frame = new JFrame("RadioButtonDemo");
46         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
47
48         frame.getContentPane().add(new RadioButtonDemo(),
49             BorderLayout.CENTER);
50         frame.pack();

```

```

51         frame.setVisible(true);
52     }
53     // هذه الدالة تُستدعى عندما يُضغط الزر Go يتم التحقق من الزر الذي تم تحديده (المختار) وطباعة رسالة ومن ثم يتم إنهاء البرنامج باستخدام System.exit(0).
54     public void actionPerformed(ActionEvent e) {
55         if (birdButton.isSelected()) {
56             System.out.println("User finally selected bird.");
57             System.exit(0);
58         }
59         if (catButton.isSelected()) {
60             System.out.println("User finally selected cat.");
61             System.exit(0);
62         }
63         if (dogButton.isSelected()) {
64             System.out.println("User finally selected dog.");
65             System.exit(0);
66         }
67         if (rabbitButton.isSelected()) {
68             System.out.println("User finally selected rabbit.");
69             System.exit(0);
70         }
71         if (pigButton.isSelected()) {
72             System.out.println("User finally selected pig.");
73             System.exit(0);
74         }
75     }

```



JComboBox:

A combo box is a button that when pressed, presents a list of items that can be selected.

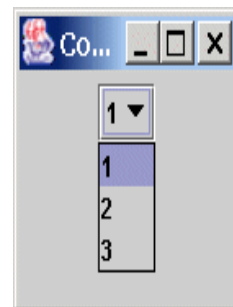
تعتبر JComboBox مكون واجهة مستخدم يتيح للمستخدم اختيار عنصر واحد فقط من القائمة المنسدلة، وهي عبارة عن زر عند الضغط عليه يتم عرض قائمة من العناصر التي يمكن اختيارها.

■ مثال:

```

1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.ActionEvent;
4  import java.awt.event.ActionListener;
5  import java.util.TreeSet;
6  import java.util.Set;
7
8  public class ComboBoxExample
implements ActionListener {
9      // تعريف متغير box من نوع JComboBox
10     JComboBox box;
11     public ComboBoxExample() {
12         JFrame frame = new JFrame
("ComboBoxExample");
13         JPanel panel = new JPanel();
14         Set s = new TreeSet();
15         // Integer يمكننا من التعامل مع العدد كـ
reference
16         s.add(new Integer(1));
17         s.add(new Integer(2));
18         s.add(new Integer(3));
19         box = new JComboBox(s.toArray());
20         box.addActionListener(this);
21         panel.add(box);
22     frame.getContentPane().add(panel);
23         frame.pack();
24         frame.setVisible(true);
25     }
26     public static void main(String[] args)
{
27         new ComboBoxExample();
28     }
29     public void actionPerformed
(ActionEvent e) {
30         //
31         if ("comboBoxChanged"
.equals(e.getActionCommand())) {
32             //
33             System.out.println("User
chose index " + box.getSelectedIndex());
34         }}}

```



■ Notice the use of the **Set**:

If we were to get the combo box choices from a file, we could prohibit duplicates by using a Set.

من الممكن استخدام الـ Set لتخزين العناصر، وهذا يضمن عدم وجود تكرار لها وخاصة عند القراءة من الملفات.

■ ملاحظة:

كما تعلمنا سابقاً، الـ Wrapper classes هي كلاسات تستخدم لتغليف المتحولات البدائية (جعل المتحولات الـ primitive كائنات من الكلاس المقابل لها).
من أمثلتها الـ Integer الذي استخدمناه في المثال السابق، ونميز:
Autoboxing: من primitive إلى reference.
Unboxing: من reference إلى primitive.

Dialogs – JOptionPane:

Dialogs are windows that are more limited than frames.

أي أن مربعات الحوار توفر واجهات بسيطة للتفاعل مع المستخدم، وهي نوافذ ثانوية أكثر محدودية من الـ frame ، وتستخدم عادة لعرض معلومات معينة (رسائل خطأ) أو لطلب إدخال من المستخدم (ولكن هذه الطريقة غير مرغوبة نوعاً ما).

Every dialog is dependent on a frame. When that frame is destroyed, so are its dependent dialogs. When the frame is iconified, its dependent dialogs disappear from the screen. When the frame is deiconified, its dependent dialogs return to the screen.

مربعات الحوار تتبع بشكل عام للنافذة الرئيسية (frame) وفي حالة إغلاق الـ frame فإن جميع مربعات الحوار المرتبطة به ستختفي أيضاً.

To create simple dialogs, use the `JOptionPane` class.

The dialogs that `JOptionPane` provides are *modal*.

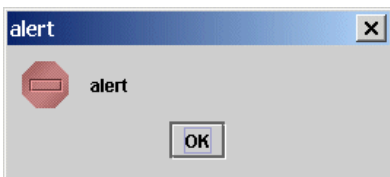
When a modal dialog is visible, it blocks user input to all other windows in the program.

أي أن مربعات الحوار التي يوفرها الكلاس `JOptionPane` تمنع المستخدم من التفاعل مع أي نوافذ أخرى في البرنامج حتى يتم إغلاقها.

■ مثال 1:

```
JOptionPane.showMessageDialog(
    null, "alert", "alert",
    JOptionPane.ERROR_MESSAGE);
```

عرض نافذة حوارية تحتوي على رسالة "alert" مع رمز خطأ، يمكن للمستخدم الضغط على زر ok لإغلاق النافذة.
`showMessageDialog`: هذه الميثود تستخدم لعرض نافذة حوارية تحتوي على رسالة.



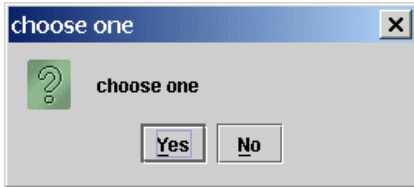
الباراميتر الأول: null وتعني أنه لا يوجد frame ، والرسالة ستظهر في منتصف الـ frame.

الباراميتر الثاني الرسالة التي سيتم عرضها في نافذة الحوار: alert
الباراميتر الثالث: alert هو عنوان نافذة الحوار.

الباراميتر الرابع: هذا يُحدد نوع الرسالة، وفي هذه الحالة، يتم عرض رمز خطأ (Error Message).

■ مثال 2 :

```
int x = JOptionPane.showConfirmDialog(null,
"choose one", "choose one",
JOptionPane.YES_NO_OPTION);
System.out.println("User clicked button " + x);
```

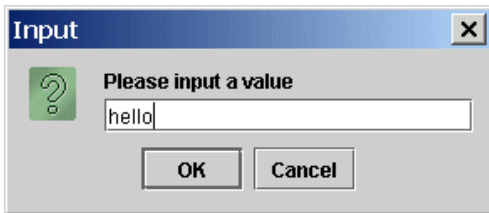


showConfirmDialog : تستخدم لعرض نافذة حوارية تسأل المستخدم سؤالاً وتقدم خيارات مثل (لا أو نعم). القيمة المعادة (x) ستعتمد على الزر الذي يضغط عليه المستخدم.

إذا ضغط المستخدم على "yes" ستعاد القيمة JOptionPane.YES_OPTION
إذا ضغط المستخدم على "NO" ستعاد القيمة JOptionPane.NO_OPTION

■ مثال 3:

```
String inputValue = JOptionPane.showInputDialog
("Please input a value"); System.out.println("User
entered " + inputValue);
```



showInputDialog : تُستخدم لعرض نافذة حوارية تطلب من المستخدم إدخال قيمة
inputValue : هذا المتغير ستُخزن فيها القيمة المدخلة

"Please input a value" : هذه هي الرسالة المعروضة في النافذة، تطلب من المستخدم إدخال قيمة.

إذا قام المستخدم بإدخال قيمة وضغط على زر "OK" سيتم طباعة القيمة المدخلة، أما إذا ضغط على زر "Cancel"، ستُعاد القيمة null .

Layout Manager:

هو مفهوم في جافا يستخدم لتنظيم وتخطيط العناصر، من اهم وظائفه :

(1) تحديد كيفية ترتيب العناصر داخل حاوية معينة، مما يساعد على إنشاء تصميم متناسق.

(2) التكيف مع الأحجام المختلفة : أي يمكن أن يتكيف Layout Manager مع تغييرات حجم النافذة مما يضمن أن تبقى العناصر مرتبة بشكل جيد.

(3) توزيع المساحة : يحدد كيفية توزيع المساحة بين العناصر.

(4) هناك 6 layout managers شائعة، وهي عبارة عن كلاسات: BorderLayout – BoxLayout – CardLayout – GridLayout – FlowLayout – GridBagLayout

1) FlowLayout:

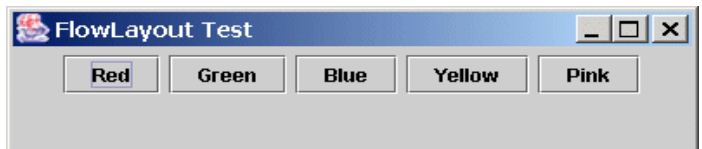
- Components are placed in a row from left to right in the order in which they are added and a new row is started when no more components can fit in the current row.
أي في FlowLayout يتم وضع المكونات من اليسار إلى اليمين وفقاً لترتيب إضافتها، ويبدأ صف جديد عندما لا يمكن أن تتسع المزيد من المكونات في الصف الحالي.
 - The components are centered in each row by default.
أي أن المكونات داخل الـ container تكون بمحاذاة الوسط، مما يعني أن المسافة من الجانبين الأيسر والأيمن متساويتان.
 - The programmer can specify the size of both the vertical and horizontal gaps between the components
أي يمكن للمبرمج أن يحدد مقدار المسافة الفاصلة بين مكون وآخر باستخدام ميثود معينة
 - FlowLayout is the default layout for JPanels.
- إذا لم يتم تحديد الـ layout فإنها تلقائياً ستكون FlowLayout، مما يسهل إضافة مكونات جديدة دون الحاجة إلى ضبط التخطيط بشكل يدوي.

■ تعريف FlowLayout: `FlowLayout f=new FlowLayout;`
`container.setLayout(f);`

■ مثال:

```

1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.ActionEvent;
4  import java.awt.event.ActionListener;
5  public class FlowLayoutTest extends JFrame{
6      //إنشاء خمسة أزرار (JButton) بأسماء مختلفة
7      //تمثل ألواناً مختلفة. كل زر يمثل لوناً معيناً
8      JButton b1=new JButton("Red"),
9      b2=new JButton("Green"),
10     b3=new JButton("Blue"),
11     b4=new JButton("Yellow"),
12     b5=new JButton("Pink");
13
14     public FlowLayoutTest() {
15         setTitle("FlowLayout Test");
16         Container pane = getContentPane();
17         //جعل الـ panel FlowLayout مما يعني
18         //أن الأزرار ستُرتب أفقياً
19         pane.setLayout(new FlowLayout());
20
21         //تحديد موضع وحجم النافذة
22         setBounds(0,0,400,100);
23         pane.add(b1);
24         pane.add(b2);
25         pane.add(b3);
26         pane.add(b4);
27         pane.add(b5);
28     }
29
30     public static void main(String args[]) {
31         JFrame f = new FlowLayoutTest();
32         f.setVisible(true);
33     }
34 }
```



2) BorderLayout:

1. Defines five locations where a component or components can be added:

North, South, East, West, and Center

يمكن إضافة المكونات في BorderLayout إلى الخمسة مواقع التي تم ذكرها، وهي ثابتة، ويمكن إضافة مكون واحد فقط في كل منطقة، وإذا قمنا بإضافة مكون إلى منطقة تحتوي بالفعل على مكون فسيتم استبداله تلقائياً.

2. The programmer specifies the area in which a component should appear
 3. The relative dimensions of the areas are governed by the size of the components added to them
- 2+3 أي يحدد المبرمج المنطقة التي يجب أن يظهر فيها المكون، في حين أن الأبعاد النسبية لهذه المناطق تعتمد على حجم المكونات المضافة إليه.

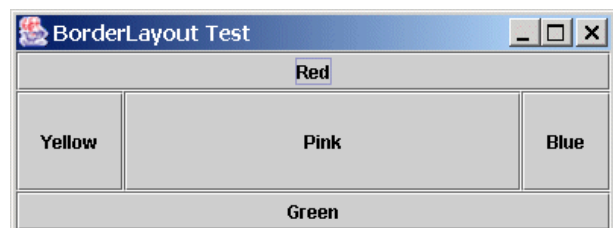


```

1  import javax.swing.*;
2  import java.awt.*;
3  import java.awt.event.ActionEvent;
4  import java.awt.event.ActionListener;
5      public class BorderLayoutTest
6  extends JFrame {
7      JButton b1=new JButton("Red"),
8      b2=new JButton("Green"),
9      b3=new JButton("Blue"),
10     b4=new JButton("Yellow"),
11     b5=new JButton("Pink");
12     public BorderLayoutTest() {
13         setTitle("BorderLayout Test");
14         Container pane = getContentPane();
15         //جعل تخطيط ال panel هو BorderLayout
16         مما يؤدي الى تقسيم النافذة إلى خمسة مناطق
17         North, South, East, Center, West
18         pane.setLayout(new BorderLayout());
  
```

```

16     setBounds(0,0,400,150);
17     //إضافة الأزرار الى pane في المناطق المحددة
18     pane.add(b1,"North");
19     pane.add(b2,"South");
20     pane.add(b3,"East");
21     pane.add(b4,"West");
22     pane.add(b5,"Center");
23 }
24 public static void main(String args[]) {
25     JFrame f = new BorderLayoutTest();
26     f.setVisible(true);
27 }
  
```



3) GridLayout:

- Components are placed in a grid with a user-specified number of columns and rows.
في هذا النوع من الـ layout يتم وضع المكونات في شبكة تتكون من عدد محدد من الأعمدة والصفوف يحدده المستخدم.
- Each component occupies exactly one grid cell.
أي أن كل عنصر من عناصر واجهة المستخدم (مثل الزر أو مربع الإدخال) يشغل مساحة خلية (تقاطع عمودين و صفين) واحدة في الشبكة فقط.
- Grid cells are filled left to right and top to bottom.
حيث يتم ملء الخلايا بالشكل الذي يبدأ من الخانة الأولى في الصف الأول، ثم ينتقل إلى الخانة التالية في نفس الصف، وعند الانتهاء من صف ما، ينتقل إلى الصف التالي (باختصار خلايا الشبكة يتم ملؤها من اليسار إلى اليمين ومن الأعلى إلى الأسفل)
- All cells in the grid are the same size.
أي أن حجم كل خلية في الشبكة متساوي، مما يساعد على الحصول على مظهر منظم ومتناسق
- Specifying zero for either rows or columns means any number of items can be placed in that row or column.
إذا تم تحديد عدد الصفوف أو الأعمدة بـ 0، فهذا يعني أنه يمكن وضع أي عدد من العناصر في هذا الصف أو العمود. بمعنى آخر لا يوجد قيود عددية على عدد العناصر التي يمكن أن تحتل الخلايا

```

1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.ActionEvent;
4 import java.awt.event.ActionListener;
5 public class GridLayoutTest extends
JFrame {
6     JButton b1=new JButton("Red"),
7         b2=new JButton("Green"),
8         b3=new JButton("Blue"),
9         b4=new JButton("Yellow"),
10        b5=new JButton("Pink");
11     public GridLayoutTest() {
12         setTitle("GridLayout Test");
13 // هذه الدالة تستخدم للوصول الى المنطقة التي
14 // يمكننا فيها اضافة مكونات الـ panel باستخدام
15 // الدالة add
16     Container pane = getContentPane();

```

```

15 // جعل تخطيط الـ GridLayout مكونة من
16 // 3 صفوف و 3 أعمدة أي أن الأزرار ستوزع في شبكة مكونة من
17 // 3 صفوف و 3 أعمدة.
18     setBounds(0,0,300,100);
19     pane.setLayout(new GridLayout(2,3));
20     pane.add(b1);
21     pane.add(b2);
22     pane.add(b3);
23     pane.add(b4);
24     pane.add(b5); }
25     public static void main(String args[]) {
26         JFrame f = new GridLayoutTest();
27         f.setVisible(true); }

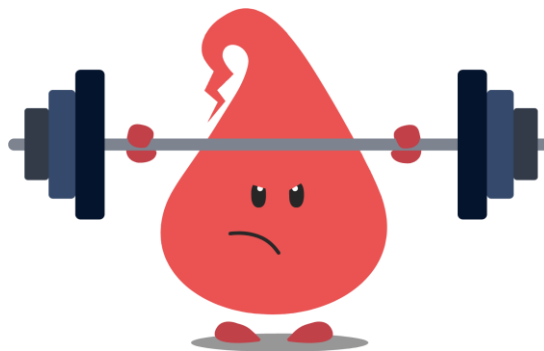
```



4) CardLayout:

هو نوع من الـ Layout Managers وسنذكر أهم النقاط التي تتعلق به :

- Components governed by a card layout are "stacked" such that only one component is displayed on the screen at any one time.
أي يتم استخدام طريقة stacked (تكديس المكونات) حيث أن جميع المكونات التي تمت إضافتها إلى container تكون موجودة، ولكن فقط مكون واحد يظهر على الشاشة في تلك اللحظة
- Components are ordered according to the order in which they were added to the container.
أي يتم ترتيب المكونات وفقاً لترتيب إضافتها إلى container . أي أن المكون الأول الذي يتم إضافته سيكون في المقدمة، والمكون الثاني سيكون خلفه، وهكذا و هذا الترتيب سيؤثر على كيفية عرض المكونات عند التبديل بينها
- Methods control which component is currently visible in the container.
- CardLayouts might be appropriate for wizards (with the Next >> buttons).
يمكن الاستفادة من CardLayouts في المعالجات wizards وخاصة التي تتطلب خطوات متعددة فعلى سبيل المثال يمكن أن تحتوي واجهة المستخدم على زر (<< Next) الذي يسمح للمستخدم بالتنقل بين الخطوات المختلفة، و كل خطوة تمثل مكوناً مختلفاً داخل CardLayout، مما يجعل التنقل بين هذه الخطوات سهلاً.



هانت يا بطل..

■ مثال عن استخدام CardLayouts:

```

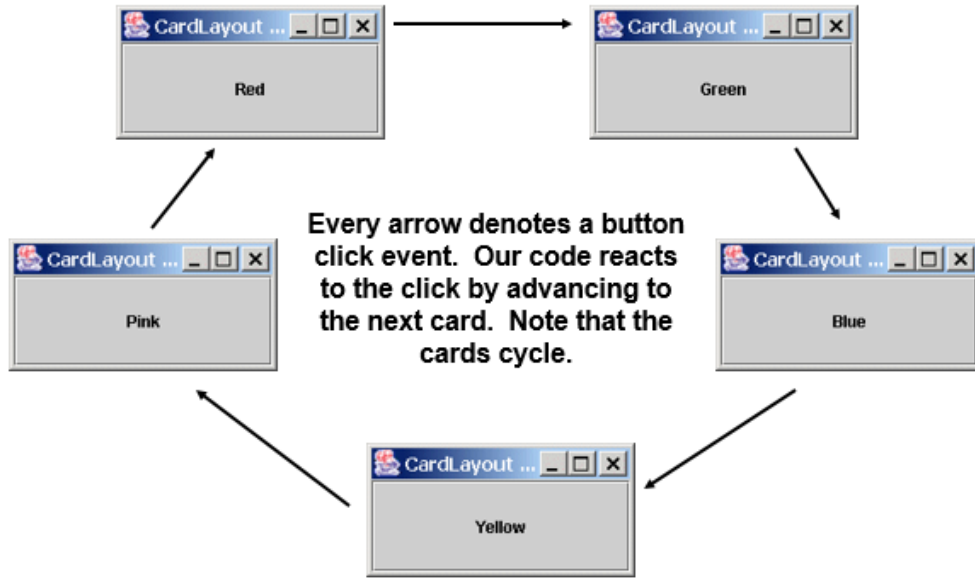
1 import javax.swing.*;
2 import java.awt.*;
3 import java.awt.event.ActionEvent;
4 import java.awt.event.ActionListener;
5 public class CardLayoutTest extends
JFrame
6 implements ActionListener {
7     JButton b1 = new JButton("Red"),
8         b2 = new JButton("Green"),
9         b3 = new JButton("Blue"),
10        b4 = new JButton("Yellow"),
11        b5 = new JButton("Pink");
12    // إنشاء كائن من CardLayout الذي
سيساعد في إدارة عرض الأزرار كبطاقات
13    CardLayout lo = new CardLayout();
14    // تعريف Container ستستخدم لتخزين الأزرار
15    Container pane;
16    public CardLayoutTest() {
17        setTitle("CardLayout Test");
18        // هذه الدالة تستخدم للوصول إلى المنطقة
التي يمكننا فيها إضافة مكونات إلى النافذة JFrame
add باستخدام الدالة (أو panel)
19        pane = getContentPane();
20        pane.setLayout(lo);
21        setBounds(0,0,200,100);
22        pane.add(b1,"1");
23        pane.add(b2,"2");
24        pane.add(b3,"3");
25        pane.add(b4,"4");
26        pane.add(b5,"5");
27        // بما ان الكلاس CardLayoutTest ينفذ
الواجهة ActionListener فيمكن استخدام الـ this
key وهي تشير الى الكائن الذي تم انشاؤه من
الكلاس CardLayoutTest وهو الذي سيتعامل مع
الحدث بدلا من انشاء كائن منه

```

```

28        b1.addActionListener(this);
29        b2.addActionListener(this);
30        b3.addActionListener(this);
31        b4.addActionListener(this);
32        b5.addActionListener(this);
33    }
34    // هذه الدالة تُستدعى عند الضغط على أي من
الأزرار
35    public void actionPerformed
(ActionEvent e) {
36        // تم استخدام lo.next(pane) لتغيير
العرض إلى البطاقة التالية في الحاوية. هذا يعني أنه
عند الضغط على أي زر، سيتم الانتقال إلى البطاقة
التالية.
37        if (e.getSource() == b1) lo.next(pane);
38        // الدالة getSource تستخدم لتحديد
المكون الذي أصدر الحدث
39        else if (e.getSource() == b2)
lo.next(pane);
40        else if (e.getSource() == b3)
lo.next(pane);
41        else if (e.getSource() == b4)
lo.next(pane);
42        else if (e.getSource() == b5)
lo.next(pane);
43    }
44    public static void main(String args[]) {
45        JFrame f = new CardLayoutTest();
46        f.setVisible(true);
47    }
48 }

```



ملاحظات اربسية :
هناك أيضاً :

■ GridBagLayout :

يستخدم لتنظيم المكونات في شبكة مع إمكانية تحديد مواقع وأحجام مختلفة لكل مكون.

■ BorderLayout :

يتيح لنا تنظيم المكونات في صفوف أفقية أو عمودية، ويتم ذلك عن طريق:

```
Boxlayout b=new Boxlayout(container, Boxlayout.x_AXIS);
Container.setlayout(b);
```

■ Null :

أي عدم الخضوع لأي تنظيم مما ذكر سابقاً واستخدام التعيين اليدوي للإحداثيات المكونات.

بعض الدروب نود أن لا تنتهي .. والسير أحياناً ألد من الوصول
إلى هنا نكون قد أنهينا مقرر البرمجة 3 لهذا الفصل
وضعنا فيه كل جهدنا وطاقاتنا راجين من الله أن نقدم لكم المعلومة
بشكل كامل، وإن أخطأنا فنحن مجرد طلاب منكم ومثلكم وجل من لا
يخطئ

لا تنسونا من صالح دعائكم

مع تحيات فريق البرمجة ☺

